DOCUMENTACION DE SISTEMA

**RootCause Bitcoin Defense**

# 02 · Instalación y ejecución

Version analizada 0.3.0 · commit acdf4dd · generado el 2026-08-27
Documento 02 de la serie · fuente: docs/system-documentation/02-installation-and-execution.md

**Contenido**


1 · Requisitos previos
2 · Instalación desde el código
3 · Ejecución en modo demostración
4 · Variables de entorno
5 · Persistencia cifrada
6 · Instalación en Windows
7 · Construir los artefactos de Windows
8 · Ejecución en contenedor
9 · Ejecución de las pruebas
10 · Conectar con Bitcoin Core
11 · Errores frecuentes durante la instalación


Todas las formas de poner en marcha el sistema, con el comando exacto y lo que cada uno verifica.

> Los comandos de este documento se probaron sobre el commit `acdf4dd` con Node v24.11.1 en Windows. Los que no se pudieron ejecutar en el entorno de análisis están marcados con [NO COMPROBADO].

---

## 1 · Requisitos previos

| Escenario | Qué hace falta |
|---|---|
| Usar la aplicación en Windows | Nada. El motor Node.js viaja dentro del paquete |
| Ejecutar desde el código | Node.js ≥ **22.12.0** (`engines` en `package.json`) |
| Contribuir o pasar las puertas de calidad | Node.js ≥ 22.12 y `git` en el PATH |
| Empaquetar para Windows | PowerShell, y para el instalador, Inno Setup 6 (`ISCC.exe`) |
| Ejecutar en contenedor | Docker con soporte de `compose.yaml` |
| Conectar con Bitcoin Core | Un nodo local con RPC habilitado |

**No hace falta `npm install` ni `pnpm install`.** El proyecto no tiene dependencias, y crear `node_modules` haría fallar la puerta `solo-local` en CI.

| Versión declarada | Dónde |
|---|---|
| Node ≥ 22.12.0 | `package.json` → `engines.node` |
| pnpm 11.19.0 | `package.json` → `packageManager` |
| Node 22 y 24 en CI | Matriz de `.github/workflows/ci.yml` |
| `node:24-alpine` | `Dockerfile` |

---

## 2 · Instalación desde el código

```
git clone https://github.com/vladimiracunadev-create/rootcause-bitcoin-defense.git
cd rootcause-bitcoin-defense
node --version
```

No hay más pasos de instalación. El siguiente comando ya arranca el sistema.

---

## 3 · Ejecución en modo demostración

Es el arranque por defecto: nada se escribe en disco y el inventario es ficticio, pero los hallazgos que produce son reales.

```
node src/server.js
```

O con el script equivalente:

```
pnpm start
```

El proceso escribe una línea JSON de arranque y queda escuchando:

```
{"level":"info","event":"server_started","address":"http://127.0.0.1:8787","mode":"demo","watchtower":false,"browserOpe
```

Abre `http://127.0.0.1:8787` en el navegador. Se detiene con `Ctrl+C`.

Para recarga automática durante el desarrollo:

```
pnpm dev
```

---

## 4 · Variables de entorno

Plantilla completa en `.env.example`. El sistema **no lee archivos `.env`**: las variables se pasan por el entorno del proceso.

| Variable | Por defecto | Qué controla |
|---|---|---|
| `HOST` | `127.0.0.1` | Interfaz de escucha. Salir de loopback dispara `BTC-OPSEC-001` |
| `PORT` | `8787` | Puerto. `0` elige uno libre, útil en pruebas |
| `DEMO_MODE` | `true` | `false` activa la persistencia cifrada |

| Variable | Por defecto | Qué controla |
|---|---|---|
| `ROOTCAUSE_DATA_KEY` | vacío | Clave AES-256. **Obligatoria** si `DEMO_MODE=false` |
| `DATA_DIR` | `./data` | Directorio del estado cifrado |
| `BITCOIN_RPC_URL` | `http://127.0.0.1:8332` | Endpoint del nodo |
| `BITCOIN_RPC_USER` | vacío | Usuario RPC. Preferible la cookie |
| `BITCOIN_RPC_PASSWORD` | vacío | Contraseña RPC. Dispara `BTC-OPSEC-005` |
| `BITCOIN_RPC_COOKIE_FILE` | vacío | Ruta al `.cookie` del nodo, la opción recomendada |
| `ALLOW_REMOTE_RPC` | `false` | Permitir un nodo fuera de loopback. Dispara `BTC-OPSEC-004` |
| `BITCOIN_RPC_TIMEOUT_MS` | `5000` | Tiempo máximo por llamada RPC |
| `WATCHTOWER_ENABLED` | `false` | Vigilancia periódica del nodo |
| `WATCHTOWER_INTERVAL_MS` | `15000` | Intervalo entre ticks, mínimo 5000 |
| `REQUEST_BODY_LIMIT_BYTES` | `65536` | Tamaño máximo de cuerpo aceptado |
| `RATE_LIMIT_PER_MINUTE` | `120` | Peticiones por minuto y origen |
| `HIGH_VALUE_THRESHOLD_SATS` | `10000000` | Umbral de «alto valor», 0,1 BTC |
| `ROOTCAUSE_OPEN_BROWSER` | sin definir | `1` abre el navegador al arrancar, solo en loopback |
| `ROOTCAUSE_CONTAINERIZED` | autodetectado | Matiza la severidad de `BTC-OPSEC-001` dentro de un contenedor |

Rangos, validación y consecuencias de configurar mal cada una en `10-configuration.md`.

---

## 5 · Persistencia cifrada

El modo demostración pierde todo al cerrar. Para conservar el inventario hacen falta dos cosas: una clave y activar el modo.

**Paso 1 — generar la clave.** Produce 32 bytes aleatorios en base64:

```
node scripts/generate-key.js
```

Salida por `stdout` (la advertencia va por `stderr`, así que se puede redirigir sin arrastrarla):

```
ROOTCAUSE_DATA_KEY=<32-bytes-aleatorios-en-base64>
```

**Paso 2 — arrancar con ella.** En PowerShell:

```
$env:DEMO_MODE = "false"
$env:ROOTCAUSE_DATA_KEY = "<pega-aqui-el-valor-generado>"
node src/server.js
```

En `bash`:

```
DEMO_MODE=false ROOTCAUSE_DATA_KEY="<pega-aqui-el-valor-generado>" node src/server.js
```

> **Si se pierde la clave, el estado cifrado no se puede recuperar.** No existe mecanismo de recuperación, y crear uno sería crear una puerta trasera. Guárdala en un gestor de contraseñas, **nunca junto al archivo de datos**.

Sin la clave, el arranque falla de forma explícita (`src/config.js` → `assertProductionConfig`):

```
ROOTCAUSE_DATA_KEY is required when DEMO_MODE=false. Run pnpm generate:data-key.
```

**Opcional — sembrar el estado cifrado con el inventario de demostración:**

```
node scripts/seed-demo.js
```

Se niega a sobrescribir un estado existente. Es una salvaguarda deliberada.

---

## 6 · Instalación en Windows

La forma normal de usar el producto. Los artefactos se publican en Releases.

| Artefacto | Para quién |
|---|---|
| `...-win-x64-setup.exe` | Instalación por usuario, sin permisos de administrador |
| `...-win-x64-portable.zip` | Descomprimir y ejecutar, sin instalar nada |

Verifica siempre la descarga contra `SHA256SUMS.txt` antes de ejecutarla:

```
Get-FileHash .\RootCause-Bitcoin-Defense-0.3.0-win-x64-setup.exe -Algorithm SHA256
```

Rutas que usa la edición de escritorio (`packaging/windows/launcher/RootCause-Bitcoin-Defense.cmd`):

| Qué | Dónde |
|---|---|
| Configuración del usuario | `%LOCALAPPDATA%\RootCause\bitcoin-defense\config.cmd` |
| Datos cifrados | `%LOCALAPPDATA%\RootCause\bitcoin-defense\data` |
| Programa | Donde se instaló, o la carpeta descomprimida |

Los datos viven **fuera** de la carpeta del programa a propósito: sobreviven a la desinstalación, porque borrar estado cifrado sin preguntar sería una pérdida irreversible. Detalles en `../WINDOWS-APP.md`.

---

## 7 · Construir los artefactos de Windows

[NO COMPROBADO] **No verificado en este análisis**: requiere PowerShell de Windows con acceso a `nodejs.org` y, para el instalador, Inno Setup 6.

```
powershell -NoProfile -ExecutionPolicy Bypass -File packaging/windows/make-icon.ps1
powershell -NoProfile -ExecutionPolicy Bypass -File packaging/windows/build-portable.ps1
```

O el script combinado:

```
pnpm package:windows
```

Lo que hace `build-portable.ps1`, en orden:

| Paso | Qué verifica |
|---|---|
| 1 | Resuelve la última LTS de la rama Node indicada |
| 2 | Descarga el ZIP oficial y **compara su SHA-256 contra `SHASUMS256.txt`** |
| 3 | Ensambla `src/`, `config/`, `scripts/`, `docs/`, `examples/` y los lanzadores |
| 4 | Escribe `BUILD-INFO.json` con versión, runtime y hash, en UTF-8 sin BOM |
| 5 | **Arranca la aplicación con el `node.exe` empaquetado** y pasa los 20 invariantes |
| 6 | Comprueba que 8 archivos obligatorios están dentro del paquete |
| 7 | Comprueba que **no** viajan `data`, `node_modules`, `.env` ni `.git` |
| 8 | Comprime, calcula el SHA-256 del ZIP y lo escribe junto a él |

Para el instalador:

```
iscc /DAppVersion=0.3.0 packaging\windows\RootCause-Bitcoin-Defense.iss
```

---

## 8 · Ejecución en contenedor

```
docker compose up --build
```

El `compose.yaml` publica en loopback y aplica un endurecimiento notable:

| Ajuste | Efecto |
|---|---|
| `ports: 127.0.0.1:8787:8787` | El puerto **no** queda expuesto a la red |
| `read_only: true` | Sistema de archivos del contenedor de solo lectura |
| `tmpfs: /tmp` | Único punto escribible efímero |
| `cap_drop: ALL` | Sin capacidades de Linux |
| `security_opt: no-new-privileges` | Sin escalada de privilegios |
| `volumes: rootcause-data` | Estado persistente fuera de la imagen |
| `USER 1000:1000` | Proceso sin root, por UID numérico |
| `HEALTHCHECK` | Golpea `/api/health` cada 30 s en forma JSON |

Dentro del contenedor `HOST=0.0.0.0` es lo normal, y el sistema lo sabe: detecta `/.dockerenv` y baja la severidad de `BTC-OPSEC-001` de crítica a media (`src/config.js` y `src/domain/posture.js`).

---

## 9 · Ejecución de las pruebas

```
node --test
```

Resultado sobre el commit analizado:

```
ℹ tests 64
ℹ pass 64
ℹ fail 0
```

Las siete puertas de calidad, en el orden en que las encadena `pnpm check`:

| # | Comando | Qué comprueba |
|---|---|---|
| 1 | `node scripts/validate-repo.js` | Archivos obligatorios, JSON válido, sintaxis, versión sincronizada |
| 2 | `node scripts/check-local-only.js` | Cero dependencias, cero orígenes remotos, CSP intacta |
| 3 | `node scripts/check-docs.js` | Todo enlace y ruta citada en la documentación existe |
| 4 | `node scripts/check-doc-style.js` | Estilo visual de la documentación |
| 5 | `node scripts/generate-rule-docs.js --check` | Las 26 fichas coinciden con el catálogo |
| 6 | `node --test` | Las 64 pruebas |
| 7 | `node scripts/check-security-claims.js` | 20 invariantes contra la aplicación en marcha |

Todo de una vez:

```
pnpm check
```

Comprobaciones que **no** están en `pnpm check` porque necesitan red o `git`:

| Comando | Requisito |
|---|---|
| `node scripts/check-history-secrets.js` | Repositorio `git` con histórico |
| `node scripts/check-runtime-advisories.js` | Acceso a la base del Node.js Security WG |
| `node scripts/audit-bitcoin.js --report` | Ambos, para cobertura completa |

---

## 10 · Conectar con Bitcoin Core

Opcional. La forma recomendada es la cookie, que el nodo rota solo:

```
BITCOIN_RPC_URL="http://127.0.0.1:8332" \
BITCOIN_RPC_COOKIE_FILE="/ruta/al/.cookie" \
WATCHTOWER_ENABLED=true \
node src/server.js
```

Para practicar sin fondos reales existe un laboratorio regtest en `labs/regtest/bitcoin.conf`, con `rpcbind` y `rpcallowip` fijados a loopback.

---

## 11 · Errores frecuentes durante la instalación

| Síntoma | Causa | Solución |
|---|---|---|
| `ROOTCAUSE_DATA_KEY is required...` | `DEMO_MODE=false` sin clave | Generar con `node scripts/generate-key.js` |
| `ROOTCAUSE_DATA_KEY must decode to exactly 32 bytes` | Clave escrita a mano o truncada | Regenerarla; solo se aceptan 32 bytes en hex o base64 |
| `EADDRINUSE` | El puerto 8787 ya está ocupado | Cambiar `PORT`, o `PORT=0` para uno libre |
| `Remote Bitcoin RPC is disabled` | `BITCOIN_RPC_URL` fuera de loopback | Usar un túnel; `ALLOW_REMOTE_RPC=true` es la última opción |
| `403 MUTATION_HEADER_REQUIRED` | Petición POST sin `x-rootcause-request: 1` | Añadir la cabecera; el panel lo hace solo |
| `415` al llamar la API | Falta `content-type: application/json` | Añadirlo |
| `Refusing to overwrite existing encrypted state` | `seed-demo.js` con estado ya presente | Es lo correcto; mover el archivo si de verdad se quiere reiniciar |
| CI falla en `solo-local` | Se creó `node_modules` o un lockfile ajeno | Borrarlos: el proyecto no instala dependencias |

Catálogo completo de problemas en `14-troubleshooting.md` y en `../TROUBLESHOOTING.md`.

---

| Siguiente | |
|---|---|
| Cómo está construido | `03-architecture.md` |
| Todas las variables, en detalle | `10-configuration.md` |
| Volver al índice | `README.md` |